

Modeling

Im Rahmen des CRISP-DM-Modells beschreibt die Phase des **Modeling** die Konzeption, Komponentenauswahl und Implementierung des KI-gestützten Agenten. Ziel dieses Kapitels ist es nicht nur die Komponentenauswahl und Architektur des Systems zu erläutern, sondern auch die zugrundeliegenden Entscheidungen und Alternativen zu reflektieren und argumentieren.

Modellverständnis und theoretische Einordnung

Einordnung des Systems im Kontext des Agentenbegriffs (nach Grundlagenkapitel)

Abgrenzung zu klassischen Machine Learning- und Data-Mining-Modellen:

Für eine klare Abgrenzung und Gegenüberstellung unseres Modelles gegenüber Machine Learning- und Data-Mining-Modellen ist es wichtig, diese zu definieren.

Klassisches Machine Learning (ML) umfasst nach (Zeigermann und Nguyen) primär drei Säulen: **Supervised Learning** (Überwachtes Lernen), **Unsupervised Learning** (Unüberwachtes Lernen) und **Reinforcement Learning** (Bestärkendes Lernen) [2]. Der gemeinsame Nenner dieser Ansätze ist ihr **analytischer** oder **prädiktiver** Charakter. Sie zielen darauf ab, Muster in Daten zu erkennen (Unsupervised), Strategien durch Interaktion zu optimieren (Reinforcement) oder Vorhersagen auf Basis gelabelter Daten zu treffen (Supervised).

Data Mining (nach Shetty et al.) bezeichnet in diesem Kontext die Anwendung von KI-Strategien auf sehr große Datenmengen (eine „enormous information base“), um darin Muster zu finden [1].

Das in dieser Arbeit modellierte System unterscheidet sich von diesen klassischen Ansätzen fundamental in Zielsetzung und Prozess:

1. **Zielsetzung (Generativ statt Prädiktiv):** Insbesondere das Supervised Learning, als am weitesten verbreitete Form des ML, ist (laut Shetty et al.) klar **prädiktiv** ausgerichtet. Das Ziel ist stets die Vorhersage eines diskreten Wertes (Klassifikation, z.B. „Spam/Kein Spam“) oder eines kontinuierlichen Wertes (Regression, z.B. ein Preis) [1]. Der in dieser Arbeit konzipierte Agent ist hingegen **generativ**. Sein Ziel ist nicht die **Vorhersage** eines einzelnen Wertes, sondern die **Erschaffung** eines neuen, komplexen und strukturierten Code-Artefakts (einer .yaml-Datei).
 2. **Prozess (Instruktion statt Training):** Klassische SML-Modelle erfordern (nach Shetty et al.) einen **gelabelten Trainingsdatensatz**, um durch einen Experten „gelehrt“ ("teaches") zu werden [1]. Der hier modellierte Agent wird nicht auf diese Weise trainiert. Er nutzt stattdessen ein vortrainiertes Fundament-Modell und wird durch **Prompt Engineering** (Instruktion) kontextualisiert, um eine spezifische Aufgabe auszuführen.
 3. **Anwendungsumfang (Spezifisch statt Explorativ):** Während Data Mining auf die explorative Analyse großer Datenmengen abzielt, operiert der Agent auf einem einzelnen, spezifischen Datenpunkt (einem GitHub-Repository), um eine eng definierte, generative Aufgabe auszuführen.
- Begründung der Einordnung als **KI-Agent** (regelbasiert, kontextadaptiv, aber nicht lernfähig)

Systemarchitektur und Prozessmodell

User Interface Layer

Repository Analyzer

AI Core Integration Layer (LLM-Client)

Pipeline Validator

Deployment Manager

- Beschreibung der Gesamtarchitektur (Microservice- / SOA-Struktur)
- Darstellung des Architekturdiagramms
- Erklärung der Komponenten **und** ihrer Rolle im **sequentiellen Prozess**:
 1. **User Interface Layer**: (Hier erklären Sie die Komponente UND ihre Prozessrolle: Eingabe der `repo_url`)
 2. **Repository Analyzer**: (Hier erklären Sie die Komponente UND ihre Prozessrolle: Wahrnehmung, Extraktion von `mta.yaml`, `package.json`...)
 3. **AI Core Integration Layer (LLM-Client)**: (Hier erklären Sie die Komponente UND ihre Prozessrolle: Kognition, Kontextaufbau, Prompt Engineering)
 4. **Pipeline Validator**: (Hier erklären Sie die Komponente UND ihre Prozessrolle: Interne Qualitätssicherung, YAML-Syntaxprüfung, Best-Practice-Analyse)
 5. **Deployment Manager**: (Hier erklären Sie die Komponente UND ihre Prozessrolle: Handlung, PR-Erstellung)
- Darstellung des Datenflusses (z.B. als Sequenzdiagramm), der diesen 5-Schritte-Prozess abbildet.

(Die alten, separaten Abschnitte “Prozessmodell und Datenfluss” und “Validierung und Qualitätsprüfung” entfallen, da ihre Inhalte jetzt logisch hier integriert sind und nicht mehr redundant erklärt werden müssen.)

Modellierungsentscheidungen und Alternativen

(Dieser Abschnitt bleibt exakt wie von Ihnen geplant, da er eine andere Ebene – das “Warum” der Tool-Auswahl – beschreibt und nicht redundant ist.)

Auswahl des Sprachmodells

- Entscheidung für **GPT-5** als vortrainiertes Large Language Model
- ... (alle Ihre Punkte bleiben) ...

Plattformwahl: SAP AI Core

- Auswahl und Integration des SAP AI Core
- ... (alle Ihre Punkte bleiben) ...

CI/CD-Plattform: GitHub Actions

- Entscheidung für GitHub Actions als Zielsystem
- ... (alle Ihre Punkte bleiben) ...

Programmiersprache und Technologie-Stack

- Wahl von **Python** als Implementierungssprache
- ... (alle Ihre Punkte bleiben) ...

Parameterisierung und Steuergrößen

- (Bleibt exakt wie von Ihnen geplant.)
- Beschreibung der Parameter **temperature** und **max_tokens**

- ...

Grenzen und Herausforderungen

- (Bleibt exakt wie von Ihnen geplant.)
- Fehlende Lernfähigkeit des Modells (kein Feedback-Loop)
- ...

Übergang zur Evaluation

- (Bleibt exakt wie von Ihnen geplant.)
- Zusammenfassung der Modellierungsergebnisse
- ...